
UNIT 4 RECURSION

Structure

- 4.0 Introduction
- 4.1 Objectives
- 4.2 Illustrative Examples
- 4.3 Analyzing Recursive Algorithms
- 4.4 Recursion Run Amok
- 4.5 Further Examples of Recursion
- 4.6 Designing Recursive Algorithms
- 4.7 Eliminating Tail Recursion
- 4.8 Summary
- 4.9 Solutions/Answers
- 4.10 Further Readings

4.0 INTRODUCTION

Recursion is one of the most powerful and elegant problem-solving techniques in computer science and programming. At its core, recursion is a method of solving problems by breaking them down into smaller, similar subproblems. A recursive solution involves a function calling itself with modified parameters until it reaches a base case that can be solved directly.

The concept of recursion appears naturally in many areas of mathematics and computer science. From calculating factorials and Fibonacci sequences to traversing tree structures and solving complex algorithmic problems, recursion provides an intuitive way to express solutions that would otherwise require complex iterative logic.

The following are key characteristics of Recursion:

- **Self-reference:** A function calls itself
- **Problem decomposition:** Large problems are broken into smaller, similar problems
- **Base case:** A condition that stops the recursive calls
- **Recursive case:** The part where the function calls itself with modified parameters

Understanding recursion is crucial for mastering advanced data structures like trees and graphs, as well as algorithmic techniques such as divide-and-conquer, dynamic programming, and backtracking.

4.1 OBJECTIVES

After studying this unit, you should be able to :

- Define and explain the concept of recursion and its fundamental principles
- Identify the essential components of a recursive algorithm (base case and recursive case)
- Trace through recursive function calls using stack diagrams and execution trees
- Analyze the time and space complexity of recursive algorithms
- Recognize when recursion can lead to inefficient solutions (recursion run amok)
- Design recursive algorithms for various problem domains

- Convert recursive solutions to iterative ones and understand tail recursion elimination
- Apply recursion to solve real-world problems including mathematical computations, data structure operations, and algorithmic challenges
- Debug recursive functions and avoid common pitfalls
- Compare recursive and iterative approaches to determine the most appropriate solution method

4.2 ILLUSTRATIVE EXAMPLES

The following are some of the examples of Recursion:

4.2.1 Mathematical Recursion: Factorial

The factorial function is the classic introduction to recursion. The factorial of a non-negative integer n (denoted as $n!$) is the product of all positive integers less than or equal to n .

Mathematical Definition:

- $0! = 1$ (base case)
- $n! = n \times (n-1)!$ for $n > 0$ (recursive case)

Recursive Implementation:

```
def factorial(n):
    # Base case
    if n == 0 or n == 1:
        return 1
    # Recursive case
    else:
        return n * factorial(n - 1)

# Example usage
print(factorial(5)) # Output: 120
print(factorial(0)) # Output: 1
```

Execution Trace for factorial(4):

```
factorial(4)
└─ 4 * factorial(3)
   └─ 4 * 3 * factorial(2)
      └─ 4 * 3 * 2 * factorial(1)
         └─ 4 * 3 * 2 * 1
            └─ 4 * 3 * 2
               └─ 4 * 6
                  └─ 24
```

4.2.2 Fibonacci Sequence

The Fibonacci sequence is another classic example where each number is the sum of the two preceding numbers.

Mathematical Definition:

- $F(0) = 0, F(1) = 1$ (base cases)
- $F(n) = F(n-1) + F(n-2)$ for $n > 1$ (recursive case)

Recursive Implementation:

```
def fibonacci(n):
    # Base cases
    if n == 0:
        return 0
    elif n == 1:
```

```

    return 1
# Recursive case
else:
    return fibonacci(n - 1) + fibonacci(n - 2)

# Example usage
print(fibonacci(6)) # Output: 8
print(fibonacci(10)) # Output: 55

```

4.2.3 String Operations: Palindrome Check

```

def is_palindrome(s):
    # Base cases
    if len(s) <= 1:
        return True
    # Recursive case
    if s[0] == s[-1]:
        return is_palindrome(s[1:-1])
    else:
        return False

# Example usage
print(is_palindrome("racecar")) # Output: True
print(is_palindrome("hello")) # Output: False

```

4.2.4 Array Operations: Sum of Elements

```

def array_sum(arr, n):
    # Base case
    if n <= 0:
        return 0
    # Recursive case
    return array_sum(arr, n - 1) + arr[n - 1]

# Example usage
numbers = [1, 2, 3, 4, 5]
print(array_sum(numbers, len(numbers))) # Output: 15

```

4.3 ANALYZING RECURSIVE ALGORITHMS

This section illustrates the process of analyzing recursive algorithms by example.

4.3.1 Call Stack Visualization

When a recursive function is called, each function call is placed on the call stack. Understanding this stack behavior is crucial for analyzing recursive algorithms.

Stack Diagram for factorial(4):

factorial(1)	<- Top of stack (returns 1)
factorial(2)	
factorial(3)	
factorial(4)	<- Bottom of stack (original call)

4.3.2 Time Complexity Analysis

Linear Recursion (Factorial):

- $T(n) = T(n-1) + O(1)$
- $T(1) = O(1)$
- Solution: $T(n) = O(n)$

Binary Recursion (Fibonacci - Naive):

- $T(n) = T(n-1) + T(n-2) + O(1)$
- $T(0) = T(1) = O(1)$
- Solution: $T(n) = O(\varphi^n)$ where $\varphi \approx 1.618$ (golden ratio)

4.3.3 Space Complexity Analysis

Space complexity is determined by the maximum depth of recursion (call stack depth):

- **Factorial:** $O(n)$ space due to n recursive calls on the stack
- **Fibonacci (naive):** $O(n)$ space due to maximum recursion depth
- **Binary tree traversal:** $O(h)$ space where h is the height of the tree

4.3.4 Recurrence Relations

Master Theorem for analyzing divide-and-conquer recurrences: For $T(n) = aT(n/b) + f(n)$:

1. If $f(n) = O(n^{\log_b(a)-\epsilon})$ for $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b(a)})$
2. If $f(n) = \Theta(n^{\log_b(a)})$, then $T(n) = \Theta(n^{\log_b(a)} \times \log n)$
3. If $f(n) = \Omega(n^{\log_b(a)+\epsilon})$ for $\epsilon > 0$, then $T(n) = \Theta(f(n))$

4.4 RECURSION RUN AMOK

4.4.1 Exponential Time Complexity

The naive Fibonacci implementation demonstrates how recursion can lead to extremely inefficient solutions:

```
def fibonacci_naive(n):
```

```
if n <= 1:
```

```
return n
```

```
return fibonacci_naive(n-1) + fibonacci_naive(n-2)
```

Problems with this approach:

- Time complexity: $O(2^n)$
- Many subproblems are solved multiple times
- For fibonacci naive(40), approximately 2^{40} function calls are made

Call tree for fibonacci(5):

```

      fib(5)
     /  \
    fib(4) fib(3)
   /  \ /  \
  fib(3) fib(2) fib(2) fib(1)
 /  \ /  \ /  \
fib(2) fib(1) fib(1) fib(0) fib(1) fib(0)
/  \
fib(1) fib(0)

```

4.4.2 Stack Overflow

Deep recursion can lead to stack overflow errors:

```
def infinite_recursion(n):
    print(f'Call {n}')
```

```
return infinite_recursion(n + 1) # No base case!

# This will cause a stack overflow
# infinite_recursion(1)
```

4.4.3 Optimizing Recursive Solutions

Memoization - storing previously computed results:

```
def fibonacci_memo(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fibonacci_memo(n-1, memo) + fibonacci_memo(n-2, memo)
    return memo[n]
```

Time complexity: $O(n)$, Space complexity: $O(n)$

Dynamic Programming - bottom-up approach:

```
def fibonacci_dp(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

4.5 FURTHER EXAMPLES OF RECURSION

The following are some more examples of Recursion:

4.5.1 Tree Traversals

Binary Tree Structure:

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

Inorder Traversal (Left, Root, Right):

Algorithm Steps:

1. Check if the current node is empty (None)
2. If empty, return an empty list (base case)
3. Otherwise, create an empty result list
4. Recursively traverse the left subtree and add its values to result
5. Add the current node's value to result
6. Recursively traverse the right subtree and add its values to result
7. Return the complete result list

Inorder Traversal (Left, Root, Right):

```
def inorder_traversal(root):
    if root is None:
        return []
    result = []
```

```

result.extend(inorder_traversal(root.left))
result.append(root.val)
result.extend(inorder_traversal(root.right))
return result

```

Preorder Algorithm Steps:

1. Check if the current node is empty (None)
2. If empty, return an empty list (base case)
3. Otherwise, create a result list starting with the current node's value
4. Recursively traverse the left subtree and add its values to result
5. Recursively traverse the right subtree and add its values to result
6. Return the complete result list

Preorder Traversal (Root, Left, Right):

```

def preorder_traversal(root):
    if root is None:
        return []
    result = [root.val]
    result.extend(preorder_traversal(root.left))
    result.extend(preorder_traversal(root.right))
    return result

```

4.5.2 Divide and Conquer: Merge Sort

Merge Sort Algorithm Steps:

1. Check if array has 1 or fewer elements (base case - already sorted)
2. If base case, return the array as is
3. Otherwise, divide the array into two halves at the middle point
4. Recursively sort the left half
5. Recursively sort the right half
6. Merge the two sorted halves back together
7. Return the merged, sorted array

Merge Function Algorithm Steps:

1. Create an empty result array and two pointers for left and right arrays
2. Compare elements at current pointers in both arrays
3. Add the smaller element to result and advance its pointer
4. Repeat until one array is exhausted
5. Add any remaining elements from the non-empty array to result
6. Return the merged result

```

def merge_sort(arr):
    # Base case
    if len(arr) <= 1:
        return arr

    # Divide
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    # Conquer
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

```

```

while i < len(left) and j < len(right):
    if left[i] <= right[j]:
        result.append(left[i])
        i += 1
    else:
        result.append(right[j])
        j += 1

```

```

result.extend(left[j:])
result.extend(right[j:])
return result

```

```

# Example usage
numbers = [64, 34, 25, 12, 22, 11, 90]
sorted_numbers = merge_sort(numbers)
print(sorted_numbers) # Output: [11, 12, 22, 25, 34, 64, 90]

```

4.5.3 Backtracking: N-Queens Problem

N-Queens Problem Algorithm Steps:

1. Initialize an empty solution list to store all valid arrangements
2. Start placing queens row by row from the first row
3. For each row, try placing a queen in each column
4. Before placing a queen, check if the position is safe (no conflicts)
5. If safe, place the queen and recursively try to place queens in remaining rows
6. If all queens are placed successfully, add the solution to the list
7. If placement leads to no solution, backtrack and try the next position
8. Continue until all possibilities are explored

Safety Check Algorithm Steps:

1. Check if any queen in previous rows is in the same column
2. Check if any queen is on the same diagonal (both directions)
3. A position is safe only if no conflicts exist with previously placed queens

```

def solve_n_queens(n):
    def is_safe(board, row, col):
        # Check column
        for i in range(row):
            if board[i] == col:
                return False

        # Check diagonals
        for i in range(row):
            if abs(board[i] - col) == abs(i - row):
                return False

        return True

    def backtrack(board, row):
        if row == n:
            solutions.append(board[:])
            return

        for col in range(n):
            if is_safe(board, row, col):
                board[row] = col
                backtrack(board, row + 1)
        # No need to reset as we overwrite in the next iteration

```

```

solutions = []
backtrack([-1] * n, 0)
return solutions

# Find all solutions for 4-Queens
solutions = solve_n_queens(4)
print(f'Number of solutions for 4-Queens: {len(solutions)}')

```

4.5.4 Graph Traversal: Depth-First Search

Depth-First Search (DFS) Algorithm Steps:

1. Initialize a set to keep track of visited vertices (avoid infinite loops)
2. Mark the starting vertex as visited
3. Process the current vertex (print, store, or perform required operation)
4. For each unvisited neighbor of the current vertex: a. Recursively call DFS on that neighbor b. The recursion will explore as deep as possible before backtracking
5. When no more unvisited neighbors exist, the function returns (backtracking)
6. The algorithm continues until all reachable vertices are visited

```

def dfs_recursive(graph, vertex, visited=None):
    if visited is None:
        visited = set()

    visited.add(vertex)
    print(vertex, end=' ')

    for neighbor in graph[vertex]:
        if neighbor not in visited:
            dfs_recursive(graph, neighbor, visited)

    return visited

# Example graph representation (adjacency list)
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

dfs_recursive(graph, 'A') # Output: A B D E F C

```

4.6 DESIGNING RECURSIVE ALGORITHMS

In this section, the process of designing recursive algorithms shall be discussed:

4.6.1 The Recursive Design Process

Step 1: Identify the Base Case(s)

Algorithm for Finding Base Cases:

1. Ask the Fundamental Question: "What is the simplest version of this problem that I can solve directly?"
2. Consider Edge Cases: Think about scenarios where:

- Input size is 0 or 1
- Range becomes empty or invalid
- Target condition is already met
- 3. Ensure Termination: The base case must guarantee that recursion stops
- 4. Provide Direct Solution: Return an immediate answer without further recursion
- 5. Examples of Common Base Cases:
 - Empty collection: return default value (0 for sum, 1 for product, empty list for concatenation)
 - Single element: return that element or simple transformation
 - Invalid range: return appropriate error value or empty result
 - Goal achieved: return success indicator or accumulated result

Step 2: Define the Recursive Case

Algorithm for Designing Recursive Case:

1. Problem Decomposition Strategy:
 - Break the problem into smaller, similar subproblems
 - Each subproblem should be of the same type as the original
 - Subproblems should be simpler (smaller size, reduced range, etc.)
2. Choose Decomposition Method:
 - Linear: Process one element, recurse on the rest ($n \rightarrow n-1$)
 - Binary: Split in half, recurse on both parts ($n \rightarrow n/2 + n/2$)
 - Multiple: Split into several parts based on problem structure
3. Define Combination Logic:
 - How do you merge results from subproblems?
 - Examples: addition, multiplication, concatenation, selection of optimal result

Step 3: Ensure Progress Toward Base Case

Algorithm for Ensuring Progress:

1. Parameter Modification Check:
 - Each recursive call must change parameters in a meaningful way
 - Changes should consistently move toward base case conditions
2. Convergence Verification:
 - Trace through a few examples manually
 - Verify that parameter values approach base case conditions
 - Ensure no infinite loops are possible
3. Common Progress Patterns:
 - Size reduction: $n \rightarrow n-1$, $n \rightarrow n/2$
 - Range narrowing: $[start, end] \rightarrow [start, mid]$ and $[mid+1, end]$
 - State advancement: moving through data structure, updating counters

Complete Design Process Example:

Problem: Calculate sum of array elements

- Step 1 - Base Case:
 - Question: "What's the simplest array sum?"
 - Answer: Empty array sums to 0
 - Base Case: if `len(arr) == 0`, return 0

- Step 2 - Recursive Case:
 - Decomposition: First element + sum of remaining elements
 - Recursive Call: `sum_array(arr[1:])`
 - Combination: `arr[0] + recursive_result`

- Step 3 - Progress Check:
 - Parameter Change: Array size decreases by 1 each call
 - Convergence: Eventually reaches empty array
 - No infinite loops possible

4.6.2 Design Patterns

Linear Recursion Pattern:

Linear Recursion Algorithm Steps:

1. Base Case Check: Determine if the problem size has reached the simplest form
 - Usually when size equals 0, 1, or some minimal condition
 - Return the direct solution for this base case
2. Problem Decomposition: Break the problem into:
 - Current element or operation
 - Remaining subproblem of size (n-1)
3. Recursive Call: Make a single recursive call with reduced problem size
 - Process one element and pass the rest to recursion
 - Trust that recursion will solve the smaller problem correctly
4. Combine Solutions: Merge the current element's contribution with the recursive result
 - This step varies based on the specific problem (sum, product, concatenation, etc.)

```
def linear_recursive_template(problem, size):  
    # Base case  
    if size == base_condition:  
        return base_solution  
  
    # Recursive case  
    subproblem_solution = linear_recursive_template(problem, size - 1)  
    return combine(current_element, subproblem_solution)
```

Binary Recursion Pattern:

Binary Recursion Algorithm Steps:

1. Base Case Check: Verify if the range is invalid or minimal
 - Usually when start $\geq$ end or size $\leq$ threshold
 - Return appropriate base solution
2. Divide Step: Split the problem into two roughly equal parts
 - Calculate midpoint to divide the range
 - Create two subproblems of approximately half the size
3. Conquer Step: Recursively solve both subproblems
 - Make two recursive calls for left and right halves
 - Each call handles its respective portion independently
4. Combine Step: Merge solutions from both subproblems
 - The combination method depends on the problem type
 - Examples: merge sorted arrays, sum values, find maximum, etc.
6. Key Advantage: Achieves $O(\log n)$ depth in recursion tree, leading to better performance for many problems.

```
def binary_recursive_template(problem, start, end):  
    # Base case  
    if start  $\geq$  end:  
        return base_solution  
  
    # Divide  
    mid = (start + end) // 2  
    left_solution = binary_recursive_template(problem, start, mid)  
    right_solution = binary_recursive_template(problem, mid + 1, end)  
  
    # Conquer  
    return combine(left_solution, right_solution)
```

4.6.3 Example: Tower of Hanoi

Problem Understanding: The Tower of Hanoi is a classic puzzle where you must move all disks from one peg to another, following these rules:

- Only one disk can be moved at a time
- A larger disk cannot be placed on top of a smaller disk
- You can use a third peg as auxiliary storage
- Tower of Hanoi Algorithm Steps:

Base Case Check: If there's only 1 disk to move:

- Simply move it directly from source to destination
- Print the move and return (no further recursion needed)

Recursive Case (3 steps for $n > 1$):

- Step 1: Move the top (n-1) smaller disks from source to auxiliary peg
 - Use destination peg as temporary storage during this process
 - This clears the way for the largest disk
- Step 2: Move the largest disk from source to destination
 - Now it's safe to move since all smaller disks are on auxiliary peg
 - Print this move
- Step 3: Move the (n-1) disks from auxiliary to destination
 - Place them on top of the largest disk
 - Use source peg as temporary storage during this process

Recursive Structure: Each recursive call solves a smaller version of the same problem, ensuring progress toward the base case.

```
def tower_of_hanoi(n, source, destination, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return

    # Move n-1 disks from source to auxiliary
    tower_of_hanoi(n-1, source, auxiliary, destination)

    # Move the largest disk from source to destination
    print(f"Move disk {n} from {source} to {destination}")

    # Move n-1 disks from auxiliary to destination
    tower_of_hanoi(n-1, auxiliary, destination, source)

# Example usage
tower_of_hanoi(3, 'A', 'C', 'B')
```

4.6.4 Example: Generate All Subsets

Problem Understanding: Generate all possible subsets (power set) of a given set. For a set with n elements, there are 2^n possible subsets, including the empty set and the set itself.

Generate All Subsets Algorithm Steps:

Main Function:

1. Initialize an empty result list to store all subsets
2. Call the backtracking helper function starting with:
 - Start index = 0 (begin with first element)
 - Empty current subset list
3. Return the complete list of all generated subsets

Backtracking Helper Function Steps:

1. Add Current State: Add a copy of the current subset to the result list
 - This represents one valid subset at this point in exploration
 - Use a copy to avoid reference issues when backtracking

2. Explore Further Choices: For each remaining element from the start index:
 - Make Choice: Add the current element to the current subset
 - Recurse: Explore all subsets that include this element
 - Increment start index to avoid duplicate combinations
 - Pass the modified current subset
 - Backtrack: Remove the element from current subset
 - This undoes the choice and prepares for the next iteration
 - Essential for exploring subsets without this element
3. Implicit Base Case: When start index reaches the end of array:
 - No more elements to consider
 - Function returns naturally (backtracking occurs)

Key Insights:

- Each element has 2 choices: include it or exclude it
- The algorithm systematically explores both choices for every element
- Backtracking ensures all combinations are generated without duplication

```
def generate_subsets(nums):
    def backtrack(start, current_subset):
        # Add current subset to result
        result.append(current_subset[:])

        # Generate subsets by including each remaining element
        for i in range(start, len(nums)):
            current_subset.append(nums[i])
            backtrack(i + 1, current_subset)
            current_subset.pop() # Backtrack

    result = []
    backtrack(0, [])
    return result

# Example usage
subsets = generate_subsets([1, 2, 3])
print(subsets) # Output: [], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]
```

4.7 ELIMINATING TAIL RECURSION

4.7.1 Understanding Tail Recursion

Tail recursion occurs when the recursive call is the last operation in the function. This is significant because tail-recursive functions can be optimized by compilers to use constant stack space.

Tail Recursive Example:

```
def factorial_tail_recursive(n, accumulator=1):
    # Base case
    if n == 0 or n == 1:
        return accumulator
    # Tail recursive case
    return factorial_tail_recursive(n - 1, n * accumulator)
```

Non-Tail Recursive (Original):

```
def factorial_non_tail(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial_non_tail(n - 1) # Multiplication happens after recursive call
```

4.7.2 Converting to Iterative Solutions

Tail Recursive to Iterative:

```
def factorial_iterative(n):
    accumulator = 1
    while n > 1:
        accumulator *= n
        n -= 1
    return accumulator
```

Fibonacci - Tail Recursive:

```
def fibonacci_tail_recursive(n, a=0, b=1):
    if n == 0:
        return a
    return fibonacci_tail_recursive(n - 1, b, a + b)
```

Fibonacci - Iterative:

```
def fibonacci_iterative(n):
    if n <= 1:
        return n

    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

4.7.3 Using Explicit Stack for General Recursion

For non-tail recursive functions, we can simulate recursion using an explicit stack:

Tree Traversal - Iterative with Stack:

```
def inorder_iterative(root):
    stack = []
    result = []
    current = root

    while stack or current:
        # Go to the leftmost node
        while current:
            stack.append(current)
            current = current.left

        # Current must be None here
        current = stack.pop()
        result.append(current.val)

        # Visit right subtree
        current = current.right

    return result
```

4.7.4 Benefits and Trade-offs

Advantages of Eliminating Recursion:

- **Constant space complexity** for tail recursion
- **Avoid stack overflow** for large inputs
- **Better performance** in some cases

Disadvantages:

- **Code complexity** may increase
- **Loss of elegance** and readability

- **Not always straightforward** to convert

4.8 SUMMARY

Recursion is a fundamental programming technique that provides elegant solutions to many complex problems. In this unit, the following are discussed:

- Recursion involves a function calling itself with modified parameters
- Every recursive function must have base cases to terminate recursion
- The call stack manages recursive function calls
- Recursion can lead to exponential time complexity if not carefully designed

The following are some of the analysis techniques:

- Time complexity analysis using recurrence relations
- Space complexity determined by recursion depth
- Master theorem for divide-and-conquer algorithms
- Recognition of when recursion becomes inefficient

The following are some of the optimization strategies:

- Memoization to avoid redundant calculations
- Tail recursion for space efficiency
- Converting recursive solutions to iterative ones
- Using explicit stacks when necessary

The following are some of the application domains:

- Mathematical computations (factorial, Fibonacci)
- Tree and graph algorithms
- Divide-and-conquer algorithms
- Backtracking problems
- String and array processing

The following are some of the best practices:

- Always define clear base cases
- Ensure progress toward base cases
- Consider iterative alternatives for simple linear recursions
- Use memoization for overlapping subproblems
- Be aware of stack overflow risks

The following guides on *when to use recursion*:

- Problem has recursive structure (trees, fractals)
- Divide-and-conquer approach is natural
- Backtracking is required
- Code clarity is significantly improved

The following guides on *when to avoid recursion* :

- Simple iterative solution exists
- Stack overflow is a concern
- Performance is critical and recursion adds overhead
- Tail recursion can be easily converted to iteration

Check Your Progress

- 1) Trace the execution of factorial(5).
- 2) Write a recursive function to find the maximum element in an array.
- 3) Calculate the time complexity of Binary Search Algorithm.
- 4) Convert the following recursive function to iterative function:

```
def sum_digits_recursive(n):  
    if n == 0:  
        return 0  
    return n % 10 + sum_digits_recursive(n // 10)
```
- 5) Implement recursive Binary Search algorithm.

4.9 SOLUTIONS/ANSWERS

1)Solution:

```
factorial(5)  
└─ 5 * factorial(4)  
   └─ 5 * 4 * factorial(3)  
      └─ 5 * 4 * 3 * factorial(2)  
         └─ 5 * 4 * 3 * 2 * factorial(1)  
            └─ 5 * 4 * 3 * 2 * 1 = 120
```

2)Solution:

```
def find_max_recursive(arr, n):  
    # Base case  
    if n == 1:  
        return arr[0]  
  
    # Recursive case  
    max_of_rest = find_max_recursive(arr, n - 1)  
    return max(arr[n - 1], max_of_rest)
```

3)Solution:

- Recurrence: $T(n) = T(n/2) + O(1)$
- Using Master Theorem: $a=1, b=2, f(n)=O(1)$
- Since $f(n) = O(n^0) = O(1)$ and $\log_2(1) = 0$
- Time complexity: $T(n) = O(\log n)$

4)Solution:

```
def sum_digits_iterative(n):  
    total = 0  
    while n > 0:  
        total += n % 10  
        n //= 10  
    return total
```

5)Solution:

```
def binary_search_recursive(arr, target, left=0, right=None):  
    if right is None:  
        right = len(arr) - 1  
  
    # Base case  
    if left > right:  
        return -1  
  
    mid = (left + right) // 2
```

```
if arr[mid] == target:
    return mid
elif arr[mid] > target:
    return binary_search_recursive(arr, target, left, mid - 1)
else:
    return binary_search_recursive(arr, target, mid + 1, right)
```

4.10 FURTHER READINGS

1. "Introduction to Algorithms" by Cormen, Leiserson, Rivest, and Stein (CLRS)
 - Comprehensive coverage of recursive algorithms and analysis
 - Chapter 4: Divide-and-Conquer
2. "The Algorithm Design Manual" by Steven S. Skiena
 - Practical approach to algorithm design
 - Extensive coverage of recursive problem-solving
3. "Algorithms" by Robert Sedgewick and Kevin Wayne
 - Clear explanations of recursive algorithms
 - Focus on implementation and analysis
4. "Programming Pearls" by Jon Bentley
 - Insights into elegant recursive solutions
 - Problem-solving techniques

Online Resources

1. MIT OpenCourseWare - Introduction to Algorithms
 - Lecture notes and videos on recursive algorithms
2. Stanford CS106B Programming Abstractions
 - Comprehensive coverage of recursion with examples
3. Visualizing Recursion - Online visualization tools
 - Interactive demonstrations of recursive algorithms
4. LeetCode and HackerRank
 - Practice problems involving recursion

Practice Problems

1. Project Euler - Mathematical problems requiring recursive solutions
2. Codeforces - Competitive programming problems
3. AtCoder - Algorithm contests with recursive challenges
4. USACO - Programming contest problems

Specialized Topics for Further Study

1. Mutual Recursion - Functions calling each other recursively
2. Continuation-Passing Style - Advanced recursive programming technique
3. Recursive Data Structures - Self-referential data structures
4. Functional Programming - Recursion as the primary control structure
5. Logic Programming - Recursion in Prolog and similar languages